1. Functional Requirements

- **Requirement: The system shall allow users to upload datasets in CSV format through the web interface.**
  Description: This is the starting point of the pipeline. The upload should show basic file details like number of rows and columns once done.

- **Requirement: The system shall let the user view a preview of the uploaded dataset before proceeding.**
  Description: A simple table view of the first few rows so the user can check if the file got read correctly.

- **Requirement: The system shall provide basic preprocessing options such as handling missing values and encoding categorical columns.**
  Description: Options can include dropping or filling null values and converting text columns to numeric using label or one-hot encoding.

- **Requirement: The system shall allow the user to select the target column for prediction.**
  Description: The user needs to mark which column is the label/output before training can start.

- **Requirement: The system shall allow the user to choose the train-test split ratio.**
  Description: A slider or input field where the user can set something like 70-30 or 80-20 split for training and testing data.

- **Requirement: The system shall provide a list of supported ML algorithms for the user to choose from.**
  Description: This includes models like Logistic Regression, Decision Tree, Random Forest, and KNN for classification, and similar options for regression.

- **Requirement: The system shall train the selected model on the uploaded dataset and show a status message once training is complete.**
  Description: Training should run in the backend and the user should get a clear success or failure message.

- **Requirement: The system shall calculate and display evaluation metrics after training, such as accuracy, precision, recall, and F1-score.**
  Description: For regression problems the system should show metrics like RMSE and R2 score instead.

- **Requirement: The system shall allow the user to compare results of two or more trained models side by side.**
  Description: This lets the user pick the best performing model for their dataset without switching between screens.

- **Requirement: The system shall let the user download the trained model file and a report of the evaluation results.**
  Description: The report can be a simple PDF summarizing the metrics and the model used.

2. Non-Functional Requirements

- **Requirement: The system should provide reasonable response time for normal user operations under college wifi.**
  Description: Applies to static pages like the home page and dashboard, not the model training step which naturally takes longer.

- **Requirement: The system should be usable without any prior ML knowledge required from the user.**
  Description: Labels, tooltips and simple instructions should be enough for a first-time user to complete a training run.

- **Requirement: The system should support multiple users without affecting the normal operation of the application.**
  Description: This is enough for demo purposes and for a small group of classmates testing it together.

- **Requirement: The system should give a proper error message instead of crashing when a bad or corrupted file is uploaded.**
  Description: For example if a user uploads an Excel file instead of CSV, the system should tell them clearly instead of showing a blank screen.

- **Requirement: The system's source code should be organized into separate modules for the frontend, backend and ML logic.**
  Description: This makes it easier to debug issues and for different team members to work on different parts without conflicts.

- **Requirement: The training and evaluation results should stay consistent when the same dataset and settings are used again.**
  Description: Random seeds should be fixed wherever the algorithm allows it, so results don't change randomly between runs.

- **Requirement: The system should be able to process datasets of a reasonable size without excessive memory usage or failure.**
  Description: This is a reasonable limit for CSV files on a student project and avoids memory issues during training.

- **Requirement: User uploaded datasets should not be visible or accessible to any other user of the system.**
  Description: Basic isolation between user sessions so people can't see each other's data.

- **Requirement: The system should work correctly on commonly used modern web browsers.**
  Description: Minor spacing differences are fine but core functionality should not break on any of these browsers.

- **Requirement: The system should be designed so that additional machine learning algorithms can be added with minimal changes to existing components.**
  Description: The model selection part should be written in a way that new models can be plugged in with minimal changes.

3. User Requirements

- **Requirement: The user should be able to create an account and log in before using the platform.**
  Description: Basic authentication so that each user's uploads and results stay separate.

- **Requirement: The user should be able to upload a dataset from their local machine without needing any technical setup.**
  Description: No installation of Python or any library should be required on the user's side.

- **Requirement: The user should be able to see what each column in the dataset looks like before choosing the target column.**
  Description: Column names, data types and a few sample values should be shown so the user can make an informed choice.

- **Requirement: The user should be able to select which machine learning algorithm to train without writing any code.**
  Description: Everything should be driven by dropdowns and buttons, since the target users are not expected to code.

- **Requirement: The user should be able to view training progress or at least know that training is currently running.**
  Description: A loading indicator or progress message so the user knows the system hasn't frozen.

- **Requirement: The user should be able to view evaluation metrics in an easy to read format such as tables or charts.**
  Description: Numbers alone can be hard to interpret, so basic bar charts or confusion matrix plots should be shown where relevant.

- **Requirement: The user should be able to re-train a model with different settings without starting over from the upload step.**
  Description: For example changing the split ratio or algorithm should not require re-uploading the same dataset again.

- **Requirement: The user should be able to download the results of their evaluation for later reference.**
  Description: Useful when the user wants to include results in a report or share with a teammate.

- **Requirement: The user should be able to manage previously generated evaluation results.**
  Description: Basic cleanup option so the user's dashboard doesn't get cluttered over time.

- **Requirement: The user should get clear feedback when something goes wrong, such as a missing target column or unsupported file type.**
  Description: Error messages should explain what went wrong in plain language, not just show a generic failure.

4. System Requirements

- **Requirement: The system shall use a web framework such as Flask or Django for the backend.**
  Description: Either is fine for this scale of project and both work well with Python based ML libraries.

- **Requirement: The system shall use a machine learning library capable of implementing the supported algorithms.**
  Description: This keeps model training consistent and avoids writing algorithms from scratch.

- **Requirement: The frontend shall be built using standard web technologies such as HTML, CSS, JavaScript, or a framework like React.**
  Description: The exact frontend stack is left to the team but should be able to talk to the backend through REST APIs.

- **Requirement: The system shall use a relational database such as MySQL or PostgreSQL to store user accounts and metadata about uploads.**
  Description: The actual dataset files can be stored separately on disk or cloud storage, with only file paths and metadata kept in the database.

- **Requirement: The system shall be deployable on a single server instance for demo purposes, such as a local server.**
  Description: The project does not need a distributed or multi-server setup given the scale expected.

- **Requirement: The system shall support modern browsers with JavaScript enabled, specifically the latest versions of Chrome, Firefox and Edge.**
  Description: Older browsers like Internet Explorer are not required to be supported.

- **Requirement: The system shall run on hardware capable of supporting the selected machine learning algorithms and dataset sizes.**
  Description: This is a reasonable assumption for typical college lab machines or basic cloud instances.

- **Requirement: The system shall communicate between frontend and backend using REST APIs returning JSON responses.**
  Description: Keeps the two parts loosely coupled and easier to test independently.

- **Requirement: The system shall provide a mechanism for storing trained models so that they can be reused without retraining.**
  Description: This allows models to be reloaded later for prediction without re-training.

5. Domain Requirements

- **Requirement: The system shall only accept structured tabular data in CSV format for training and evaluation.**
  Description: Unstructured data like images or text documents is out of scope for this project.

- **Requirement: The system shall support both classification and regression type problems based on the target column type.**
  Description: The system should automatically detect if the target is categorical or continuous and adjust the workflow accordingly.

- **Requirement: The system shall perform a train-test split before training any model, and shall not evaluate a model on data it was trained on.**
  Description: This is a basic requirement to avoid misleading accuracy numbers due to overfitting.

- **Requirement: The system shall provide standard preprocessing steps like null value handling and feature scaling before training.**
  Description: Most ML algorithms need clean, numeric input, so this step is necessary before any model can be trained properly.

- **Requirement: The system shall calculate evaluation metrics appropriate to the problem type, such as accuracy and F1-score for classification, and RMSE and R2 for regression.**
  Description: Using the wrong metric for the wrong problem type would give the user incorrect conclusions about model performance.

- **Requirement: The system shall allow comparison of at least two different algorithms on the same dataset and split.**
  Description: This is the core purpose of the tool, letting the user see which algorithm works best for their data.

- **Requirement: The system shall generate a confusion matrix for classification problems as part of the evaluation output.**
  Description: Gives the user a more detailed view of where the model is making mistakes, beyond a single accuracy number.

- **Requirement: The system shall distinguish between input features and the target variable before model training.**
  Description: The target column selected by the user shall be used as the output variable, while the remaining suitable columns shall be treated as input features.

- **Requirement: The system shall not allow training to proceed if the dataset has fewer than a minimum number of rows, for example 20.**
  Description: Training on a very small dataset would not give meaningful results and can also cause errors in the train-test split step.

- **Requirement: The system shall visually represent evaluation results using charts such as bar graphs for metric comparison.**
  Description: Helps the user quickly understand and compare model performance without going through raw numbers.